

- (4) Definire intuitivamente e formalmente (mediante definizione semantica) cosa è un interprete. Descrivere e spiegare poi la struttura di un interprete (come pseudo-codice o come flow-chart).

Un interprete è un insieme di strutture dati e algoritmi per memorizzare ed eseguire programmi. Semanticamente è una macchina astratta che esegue un linguaggio macchina e ritorna dei dati:

$$I^{L_0L} : \text{Prog}^L \times \text{Dat}^i \rightarrow \text{Dat}^o$$

$$I^{L_0L}(P^L, \text{input}) = \text{output} \quad \text{t.c.} \quad P^L(\text{input}) = \text{output}, \quad P^L \in \text{Prog}^L, \quad \text{input} \in \text{Dat}^i, \quad \text{output} \in \text{Dat}^o$$

La macchina astratta è implementata in un linguaggio di livello inferiore L_0 ed esegue il linguaggio macchina L di livello superiore. La struttura dell'interprete è la seguente:

pc = 0	1. PC
while (true) {	2. While loop
istruzione = fetch(pc)	3. Fetch
op = decode(istruzione)	4. Decode
args = null	5. Get args if needed
if op needs args {	6. Execute
args = fetch_args(op)	7. Store result if present
}	8. PC++
switch (op) {	
case op1: exec(op1, args)	
case op2: exec(op2, args)	
...	
case opn: exec(opn, args)	
case halt: return;	
}	
if opcode has result {	
store(result)	
}	
pc += 1	
}	

- (3) Si consideri l'insieme E definito induttivamente: $n \in E$, e se $e_1, e_2 \in E$ allora $e_1 + e_2 \in E$, $e_1 - e_2 \in E$. Si definiscano sugli elementi di tale insieme le seguenti funzioni: $v(n) = 1$, $v(e_1 + e_2) = v(e_1 - e_2) = v(e_1) + v(e_2)$, e $o(n) = 0$, $o(e_1 + e_2) = o(e_1 - e_2) = o(e_1) + o(e_2) + 1$. Si dimostri per induzione strutturale che $\forall e \in E. v(e) = o(e) + 1$.

$$E: \begin{cases} n \in E \\ e_1, e_2 \in E \Rightarrow e_1 + e_2 \in E \wedge e_1 - e_2 \in E \end{cases}$$

$$v: \begin{cases} v(n) = 1 \\ v(e_1 + e_2) = v(e_1 - e_2) = v(e_1) + v(e_2) \end{cases}$$

$$o: \begin{cases} o(n) = 0 \\ o(e_1 + e_2) = o(e_1 - e_2) = o(e_1) + o(e_2) + 1 \end{cases}$$

Dimostrare: $\forall e \in E. v(e) = o(e) + 1$

Dimostrazione per induzione strutturale

Caso base

$$v(n) = o(n) + 1 \Rightarrow 1 = 0 + 1 \Rightarrow 1 = 1 \quad \checkmark$$

Passo induttivo

Dato un costruttore supponiamo che valga l'ipotesi induttiva sugli elementi dell'insieme e dimostriamo che il costruttore mantiene la proprietà

$$e = e_1 + e_2$$

Valuteremo $e_1 + e_2$ senza perdita di generalità perchè $e_1 - e_2$ si comporta in maniera analoga

Hip. ind. proprietà vale su $e_1 \wedge e_2$

$$v(e_1) = o(e_1) + 1$$

$$v(e_2) = o(e_2) + 1$$

Tesi: $v(e) = o(e) + 1$

$$v(e) = v(e_1 + e_2) = v(e_1) + v(e_2)$$

$$o(e) = o(e_1 + e_2) = o(e_1) + o(e_2) + 1$$

$$\underbrace{v(e_1) + v(e_2)}_{v(e)} = \underbrace{o(e_1) + o(e_2) + 1}_{o(e)} + 1$$

$\Downarrow$ i.i.

$$o(e_1) + 1 + o(e_2) + 1 = o(e_1) + o(e_2) + 1 + 1$$

$$o(e_1) + o(e_2) + 2 = o(e_1) + o(e_2) + 2$$

$$0 = 0 \quad \checkmark$$

3. (5) Si consideri il programma sulla destra. Si dica cosa viene calcolato dall'assegnamento in caso di scoping statico (mostrando come vengono calcolati i link statici e come vengono risolti riferimenti non locali) e in caso di scoping dinamico (mostrando l'evoluzione della tabella centrale dei riferimenti (CRT) e come vengono risolti riferimenti non locali).

```
{int a=2;
void pippo(){
    a = a + 1;}
void pluto(){
    int a = 1;
    void minnie(int b){
        int a = 2*b;
        pippo();}
    pippo();
    minnie(3);}
a=3;
pluto(); }
```

Sbagliato

```

{
  int a = 2;

  void pippo() {
    a = a + 1;
  }

  void pluto() {
    int a = 1;

    void minnie(int b) {
      int a = 2*b;
      pippo();
    }

    pippo();
    minnie(3);
  }

  a = 3;
  pluto();
}

```

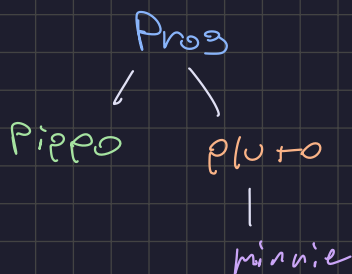
Statico

```

1 a = 2
2 a = 3
3 pluto() → a = 1
4 pippo() → a = a + 1 → a = 4
5 minnie(3) → a = 2 · b = 2 · 3 = 6
6 pippo() → a = a + 1 → a = 5
7 END
↓
a = 5

```

Relazione di definizione



Dinamico

Relazione di esecuzione

```

1 a = 2
2 a = 3
3 pluto()
4   int a = 1
5   pippo()
6     a = a + 1 → a = 2
7   minnie(3)
8     int a = 2 · 3 = 6
9     pippo()
10    a = a + 1 → a = 7
11 END
↓
a = 3

```

Prog → Pluto → Pippo → Minnie → Pippo

4. (6) Spiegare dove e perché nel programma è necessario parlare di regole di scoping, e dove (e perché) anche di regole di binding. Si dica quale risulta essere il valore finale di z nelle varie situazioni, ovvero con: (1) scoping statico; (2) scoping dinamico e deep-binding; (3) shallow-binding (e scoping dinamico).

```
{int a = 4; int b = 3;
int pippo (int in){
    return a + b - in;
}
int pluto (int f(int b)){
    int b = 5; int a = 6;
    return f(b) + a;
}
{int b = 2;
int z = pluto(pippo);
}
```

```
{
  int a = 4;
  int b = 3;

  int pippo(int in) {
    return a + b - in;
  }

  int pluto(int f(int b)) {
    int b = 5;
    int a = 6;
    return f(b) + a;
  }

  {
    int b = 2;
    int z = pluto(pippo);
  }
}
```

Si devono introdurre regole di scoping perchè le richiede la funzione pippo, siccome all'interno sono presenti variabili non locali e quindi bisogna risolvere i binding a cui fanno riferimento.

Si devono introdurre regole di binding perchè le richiede la funzione?

Sbagliato

Ordine di definizione



Scoping statico

```

1 int a = 4
2 int b = 3
3 int b = 2
4 int z = pluto(Pippo)
5   int b = 5
6   int a = 6
7   return f(b) + a → pippo(5) + 6
8     return a + b - in → 4 + 3 - 5 = 2 → return 2 + 6 = 8
9 END
  ↓
  z = 8

```

Ordine di esecuzione

Prog → Block → Pluto → Pippo → Pluto → Block → Prog

Scoping dinamico

```

1 int a = 4
2 int b = 3
3 int b = 2
4 int z = pluto(Pippo)
5   int b = 5
6   int a = 6
7   return f(b) + a → pippo(5) + 6
8     return 6 + 5 - 5
9     return 6 + 6
10 END
  ↓
  z = 12

```

6. (4) Siano $\rho = [x \mapsto 2, y \mapsto l_y]$ un ambiente dinamico con l_y locazione di tipo int, e $\sigma = [l_y \mapsto 3]$ una memoria. Descrivere le derivazioni della semantica dinamica per il comando $y := x + y$ nell'ambiente ρ a partire dalla memoria σ ($\rho \vdash \langle y := x + y, \sigma \rangle \rightarrow \dots$).

$$\rho = [x \mapsto 2, y \mapsto l_y] \quad \Delta = [l_y \mapsto \text{int loc}] \quad \sigma = [l_y \mapsto 3]$$

Suppongo che sia ben formato

$$y := x + y$$

La regola da applicare per eseguire questo comando è:

$$\frac{\rho \vdash \langle x + y, \sigma \rangle \rightarrow_e^* \langle k, \sigma \rangle}{\rho \vdash \langle y := x + y, \sigma \rangle \rightarrow_c \langle y := k, \sigma \rangle} \quad (1)$$

Per ottenere k bisogna prima di tutto valutare l'espressione $x + y$, per farlo bisogna prima valutare x e y separatamente:

$$\sim \text{Valutazione di } x \quad \rho \vdash \langle x, \sigma \rangle \rightarrow_e \langle 2, \sigma \rangle, \quad \rho[x] = 2$$

$$\sim \text{Valutazione di } y \quad \rho \vdash \langle y, \sigma \rangle \rightarrow_e^* \langle 3, \sigma \rangle, \quad \sigma[\rho[y]] = 3$$

Ora posso valutare l'espressione intera

$$\frac{\rho \vdash \langle x, \sigma \rangle \rightarrow_e \langle 2, \sigma \rangle \quad \rho \vdash \langle y, \sigma \rangle \rightarrow_e^* \langle 3, \sigma \rangle}{\rho \vdash \langle x + y, \sigma \rangle \rightarrow_e \langle 2 + 3, \sigma \rangle} \quad \rho[x] = 2 \wedge \sigma[\rho[y]] = 3$$

Applico l'assioma per calcolare il valore finale dell'espressione

$$\rho \vdash \langle 2 + 3, \sigma \rangle \rightarrow_e \langle 5, \sigma \rangle$$

Adesso posso applicare la regola (1) per l'assegnamento

$$\frac{\rho \vdash \langle x + y, \sigma \rangle \rightarrow_e^* \langle 5, \sigma \rangle}{\rho \vdash \langle y := x + y, \sigma \rangle \rightarrow_c \langle y := 5, \sigma \rangle}$$

L'esecuzione di $y := 5$ ritorna una nuova memoria σ'

$$\rho \vdash \langle y := 5, \sigma \rangle \rightarrow_c \sigma[l_y \mapsto 5] = [l_y \mapsto 5] = \sigma'$$

Secondo tentativo

2. (3) Si consideri l'insieme E definito induttivamente: $n \in E$, e se $e_1, e_2 \in E$ allora $e_1 + e_2 \in E$, $e_1 - e_2 \in E$. Si definiscano sugli elementi di tale insieme le seguenti funzioni: $v(n) = 1$, $v(e_1 + e_2) = v(e_1 - e_2) = v(e_1) + v(e_2)$, e $o(n) = 0$, $o(e_1 + e_2) = o(e_1 - e_2) = o(e_1) + o(e_2) + 1$. Si dimostri per induzione strutturale che $\forall e \in E. v(e) = o(e) + 1$.

$$n \in E, \quad e_1, e_2 \in E \rightarrow e_1 + e_2 \in E \wedge e_1 - e_2 \in E$$

$$v(n) = 1$$

$$o(n) = 0$$

$$v(e_1 + e_2) = v(e_1 - e_2) = v(e_1) + v(e_2)$$

$$o(e_1 + e_2) = o(e_1 - e_2) = o(e_1) + o(e_2) + 1$$

Dimostrare: $\forall e \in E. v(e) = o(e) + 1$

Dimostro per induzione strutturale

Caso base

$$n \in E \rightarrow v(n) = 1, \quad o(n) = 0$$

$$v(n) = 1 = 0 + 1 = o(n) + 1$$

Passo induttivo

$$\text{Hp. Induttiva: } e = e_1 + e_2 \in E \rightarrow v(e_1) = o(e_1) + 1$$

$$v(e_2) = o(e_2) + 1$$

Tesi da dimostrare $v(e) = o(e) + 1$

$$v(e) = v(e_1 + e_2)$$

$$\stackrel{\text{Hp. I}}{=} o(e_1) + 1 + o(e_2) + 1$$

$$= o(e_1) + o(e_2) + 1 + 1$$

$$= o(e) + 1$$

La dimostrazione è analoga per il -

3. (5) Si consideri il programma sulla destra. Si dica cosa viene calcolato dall'assegnamento in caso di scoping statico (mostrando come vengono calcolati i link statici e come vengono risolti riferimenti non locali) e in caso di scoping dinamico (mostrando l'evoluzione della tabella centrale dei riferimenti (CRT) e come vengono risolti riferimenti non locali).

```
{int a=2;
void pippo(){
    a = a + 1;}
void pluto(){
    int a = 1;
    void minnie(int b){
        int a = 2*b;
        pippo();}
    pippo();
    minnie(3);}
a=3;
pluto(); }
```

```

{
  int a = 2;

  void pippo() {
    a = a + 1;
  }

  void pluto() {
    int a = 1;

    void minnie(int b) {
      int a = 2*b;
      pippo();
    }

    pippo();
    minnie(3);
  }

  a = 3;
  pluto();
}

```

Catena di annidamento

```

main (a)      SD(main) = 0
├ pippo      SD(pippo) = 1
├ pluto (a)   SD(pluto) = 1
└ minnie (a)  SD(minnie) = 2

```

Catena di esecuzione

```

main → pluto → pippo
           ↘ minnie → pippo

```

Scoping statico

Esecuzione main → pluto

link statico $K = SD(\text{chiamante}) - SD(\text{chiamato}) + 1$

$K_{\text{pluto}} = SD(\text{main}) - SD(\text{pluto}) + 1$

$$= 0 - 1 + 1 = 0 \Rightarrow CS(\text{pluto}) = \text{main}$$

Esecuzione main: $a = 3$

Esecuzione pluto → pippo

$K_{\text{pippo}} = SD(\text{pluto}) - SD(\text{pippo}) + 1$

$$= 1 - 1 + 1 = 1 \Rightarrow CS(\text{pippo}) = \text{main}$$

Esecuzione pippo: $a = a + 1$

a non è locale

$N = SD(\text{uso}) - SD(\text{def})$

$N_a = SD(\text{pippo}) - SD(\text{main})$

$$= 1 - 0 = 1 \Rightarrow a = 3$$

$a = a + 1 = 3 + 1 = 4$

Terminazione di pippo: viene tolto dallo stack

Esecuzione pluto → minnie(3)

$K_{\text{minnie}} = SD(\text{pluto}) - SD(\text{minnie}) + 1$

$$= 1 - 2 + 1 = 0 \Rightarrow CS(\text{minnie}) = \text{pluto}$$

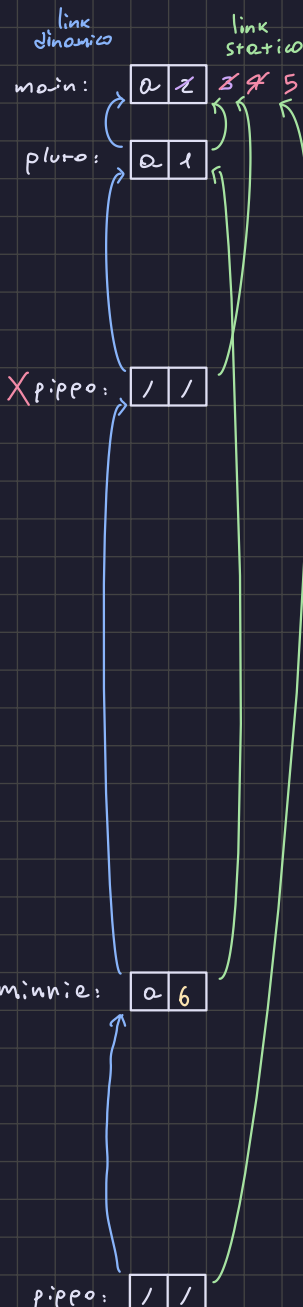
Esecuzione minnie: $\text{int } a = 2 \cdot b = 2 \cdot 3 = 6$

Esecuzione minnie → pippo $a = a + 1$

a non è locale

$K_{\text{pippo}} = SD(\text{minnie}) - SD(\text{pippo}) + 1$

$$= 2 - 1 + 1 = 2 \Rightarrow CS(\text{pippo}) = \text{main}$$



$$N_a = SD(\text{minnie}) - SD(\text{main})$$

$$= 2 \Rightarrow a = 4$$

$$a = a + 1 = 4 + 1 = 5$$

Successivamente tutte le chiamate terminano e lo stack viene deallocato

Scoping dinamico

Eseecuzione del main

$a \rightarrow \text{main}, 3$

Eseecuzione $\text{main} \rightarrow \text{pluto}$

$a \rightarrow \text{pluto}, 4 \rightarrow \text{main}, 3$

Eseecuzione $\text{pluto} \rightarrow \text{pippo}$

$$a = a + 1 = 4 + 1 = 5$$

Eseecuzione $\text{pluto} \rightarrow \text{minnie}(3)$

$a \rightarrow \text{minnie}, 6 \rightarrow \text{pluto}, 4 \rightarrow \text{main}, 3$

Eseecuzione $\text{minnie} \rightarrow \text{pippo}$

$$a = a + 1 = 6 + 1 = 7$$

4. (6) Spiegare dove e perché nel programma è necessario parlare di regole di scoping, e dove (e perché) anche di regole di binding. Si dica quale risulta essere il valore finale di z nelle varie situazioni, ovvero con: (1) scoping statico; (2) scoping dinamico e deep-binding; (3) shallow-binding (e scoping dinamico).

```
{int a = 4; int b = 3;
int pippo (int in){
    return a + b - in;
}
int pluto (int f(int b)){
    int b = 5; int a = 6;
    return f(b) + a;
}
{int b = 2;
int z = pluto(pippo);
}
```

{ 1 Ambiente di definizione

```
int a = 4;
int b = 3;
```

```
int pippo(int in) {
    return a + b - in;
}
```

```
int pluto(int f(int b)) { 3. Ambiente di esecuzione
    int b = 5;
    int a = 6;
    return f(b) + a;
}
```

{ 2. Ambiente di creazione del lego-me

```
int b = 2;
int z = pluto(pippo);
}
```

```
}
```


1. Scoping statico Ambiente 1: $a=4$ $b=3$

pluto(pippo) $\rightarrow$ pippo(5) + $a \rightarrow a + b - 5 = 2$

8 2 6 4 3

locale

$z=8$

2. Scoping dinamico + deep binding Ambiente 2 $\rightarrow b=2$ Ambiente 1 $\rightarrow a=4$

pluto(pippo) $\rightarrow$ pippo(5) + $a \rightarrow a + b - 5 = 1$

7 1 6 4 2

locale

$z=7$

2. Scoping dinamico + shallow binding Ambiente 3 $a=6$ $b=5$

pluto(pippo) $\rightarrow$ pippo(5) + $a \rightarrow a + b - 5 = 6$

12 6 6 6 5

locale

$z=12$

5. (5) Si dica per ognuno dei seguenti metodi di passaggio di parametri, quali sono i valori delle variabili dopo l'esecuzione del codice a destra: Passaggio per valore e Passaggio per riferimento. Dire quali sono i valori di x e y nel main al ritorno di pippo e descrivere dettagliatamente cosa accade nella memoria.

```
void pippo(int a, int b){
    a = 5 + a;
    b = 3b;
}
void main(){
    int x = 4, y=5;
    pippo(x,y);
}
```

Passaggio per valore

Esecuzione main

RDA
main: $x \rightarrow 4$
 $y \rightarrow 5$

Chiamata pippo

pippo: $a \rightarrow 4$
 $b \rightarrow 5$

copia

Esecuzione pippo

RDA
RDA main: $x \rightarrow 4$
 $y \rightarrow 5$

RDA pippo: $a \rightarrow 9$
 $b \rightarrow 15$

Terminazione pippo

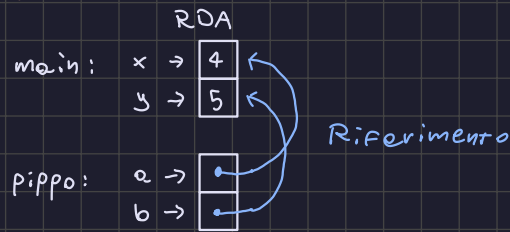
RDA
RDA main: $x \rightarrow 4$
 $y \rightarrow 5$

~~RDA pippo: $a \rightarrow 9$
 $b \rightarrow 15$~~

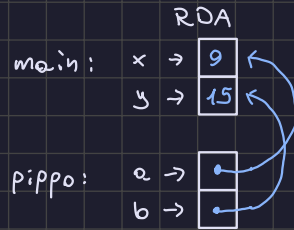
Alla terminazione di pippo $x = 4$ e $y = 5$.

Passaggio per riferimento

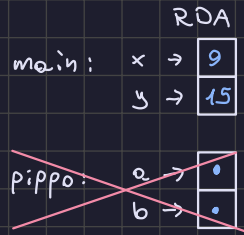
Esecuzione main



Esecuzione pippo



Terminazione pippo



Alla terminazione di pippo $x = 9$ e $y = 15$.

6. (4) Siano $\rho = [x \mapsto 2, y \mapsto l_y]$ un ambiente dinamico con l_y locazione di tipo int, e $\sigma = [l_y \mapsto 3]$ una memoria. Descrivere le derivazioni della semantica dinamica per il comando $y := x + y$ nell'ambiente ρ a partire dalla memoria σ ($\rho \vdash \langle y := x + y, \sigma \rangle \rightarrow \dots$).

$$\rho = [x \mapsto 2, y \mapsto l_y] \quad \sigma = [l_y \mapsto 3]$$

Per valutare $y := x + y$ bisogna prima valutare $x + y$ e per fare ciò bisogna prima valutare x e poi y

Valutazione di x

$$\rho \vdash \langle x, \sigma \rangle \rightarrow \langle 2, \sigma \rangle, \quad \rho(x) = 2$$

Valutazione di y

$$\rho \vdash \langle y, \sigma \rangle \rightarrow \langle 3, \sigma \rangle, \quad \rho(\sigma(l_y)) = 3$$

Ora si può valutare $x + y$

Valutazione di $x + y$

$$\frac{\rho \vdash \langle x, \sigma \rangle \rightarrow \langle 2, \sigma \rangle \quad \rho \vdash \langle y, \sigma \rangle \rightarrow \langle 3, \sigma \rangle}{\rho \vdash \langle x + y, \sigma \rangle \rightarrow \langle 3 + 2, \sigma \rangle} \quad \begin{array}{l} \rho(x) = 2 \wedge \\ \rho(\sigma(l_y)) = 3 \end{array}$$

$$\rho \vdash \langle x + y, \sigma \rangle \rightarrow \langle 5, \sigma \rangle$$

Ora si può valutare l'assegnamento $y := x + y$

Valutazione $y := x + y$

$$\rho \vdash \langle x + y, \sigma \rangle \rightarrow \langle 5, \sigma \rangle$$

$$\rho \vdash \langle y := x + y, \sigma \rangle \rightarrow \langle y := 5, \sigma \rangle$$

$$\rho \vdash \langle y := 5, \sigma \rangle \rightarrow \sigma' = [l_y = 5]$$